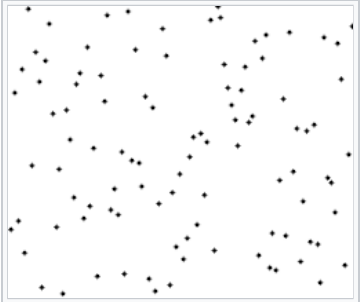
Sorting algorithm

54 languages

In computer science, a **sorting algorithm** is an algorithm that puts elements of a list into an order. The most frequently used orders are numerical order and lexicographical order, and either ascending or descending. Efficient sorting is important for optimizing the efficiency of other algorithms (such as search and merge algorithms) that require input data to be in sorted lists. Sorting is also often useful for canonicalizing data and for producing human-readable output.


Merge sort

Formally, the output of any sorting algorithm must satisfy two conditions:

1. The output is in monotonic order (each element is no smaller/larger than the previous element, according to the required order).
2. The output is a permutation (a reordering, yet retaining all of the original elements) of the input.

Although some algorithms are designed for sequential access, the highest-performing algorithms assume data is stored in a data structure which allows random access.

History and concepts [edit]

From the beginning of computing, the sorting problem has attracted a great deal of research, perhaps due to the complexity of solving it efficiently despite its simple, familiar statement. Among the authors of early sorting algorithms around 1951 was Betty Holberton, who worked on ENIAC and UNIVAC.[1][2] Bubble sort was analyzed as early as 1956.[3] Asymptotically optimal algorithms have been known since the mid-20th century – new algorithms are still being invented, with the widely used Timsort dating to 2002, and the library sort being first published in 2006.

Comparison sorting algorithms have a fundamental requirement of n log n - 1.4427n + O(log n) comparisons. Algorithms not based on comparisons, such as counting sort, can have better performance.

Sorting algorithms are prevalent in introductory computer science classes, where the abundance of algorithms for the problem provides a gentle introduction to a variety of core algorithm concepts, such as big O notation, divide-and-conquer algorithms, data structures such as heaps and binary trees, randomized algorithms, best, worst and average case analysis, time–space tradeoffs, and upper and lower bounds.

Sorting small arrays optimally (in the fewest comparisons and swaps) or fast (i.e. taking into account machine-specific details) is still an open research problem, with solutions only known for very small arrays (<20 elements). Similarly optimal (by various definitions) sorting on a parallel machine is an open research topic.

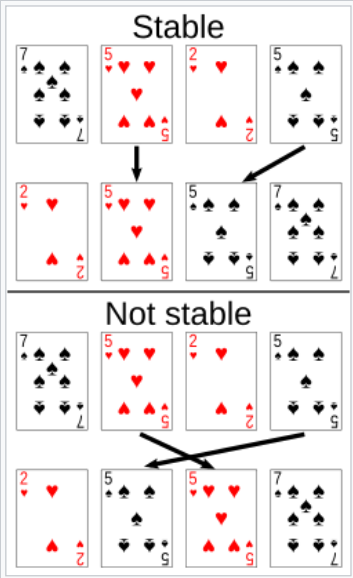
Classification [edit]

Sorting algorithms can be classified by:

- Computational complexity
  - Best, worst and average case behavior in terms of the size of the list. For typical serial sorting algorithms, good behavior is O(n log n), with parallel sort in O($\log^2 n$), and bad behavior is O(n^2). Ideal behavior for a serial sort is O(n), but this is not possible in the average case. Optimal parallel sorting is O(log n).
  - Swaps for "in-place" algorithms.
- Memory usage (and use of other computer resources). In particular, some sorting algorithms are "in-place". Strictly, an in-place sort needs only O(1) memory beyond the items being sorted; sometimes O(log n) additional memory is considered "in-place".
- Recursion: Some algorithms are either typically recursive or typically non-recursive, while others may typically be both (e.g., merge sort).
- Stability: stable sorting algorithms maintain the relative order of records with equal keys (i.e., values).
- Whether or not they are a comparison sort. A comparison sort examines the data only by comparing two elements with a comparison operator.
- General method: insertion, exchange, selection, merging, *etc.* Exchange sorts include bubble sort and quicksort. Selection sorts include cycle sort and heapsort.
- Whether the algorithm is serial or parallel. The remainder of this discussion almost exclusively concentrates on serial algorithms and assumes serial operation.
- Adaptability: Whether or not the presortedness of the input affects the running time. Algorithms that take this into account are known to be adaptive.
- Online: An algorithm such as Insertion Sort that is online can sort a constant stream of input.

Stability [edit]

Stable sorting algorithms sort equal elements in the same order that they appear in the input. For example, in the card sorting example to the right, the cards are being sorted by their rank, and their suit is being ignored. This allows the possibility of multiple different correctly sorted versions of the original list. Stable sorting algorithms choose one of these, according to the following rule: if two items compare as equal (like the two 5 cards), then their relative order will be preserved, i.e. if one comes before the other in the input, it will come before the other in the output.



An example of stable sort on playing cards. When the cards are sorted by rank with a stable sort, the two 5s must remain in the same order in the sorted output that they were originally in. When they are sorted with a non-stable sort, the 5s may end up in the opposite order in the sorted output.

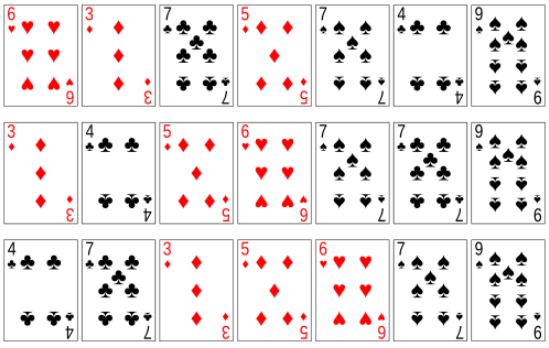
Stability is important to preserve order over multiple sorts on the same data set. For example, say that student records consisting of name and class section are sorted dynamically, first by name, then by class section. If a stable sorting algorithm is used in both cases, the sort-by-class-section operation will not change the name order; with an unstable sort, it could be that sorting by section shuffles the name order, resulting in a nonalphabetical list of students.

More formally, the data being sorted can be represented as a record or tuple of values, and the part of the data that is used for sorting is called the *key*. In the card example, cards are represented as a record (rank, suit), and the key is the rank. A sorting algorithm is stable if whenever there are two records R and S with the same key, and R appears before S in the original list, then R will always appear before S in the sorted list.

When equal elements are indistinguishable, such as with integers, or more generally, any data where the entire element is the key, stability is not an issue. Stability is also not an issue if all keys are different.

Unstable sorting algorithms can be specially implemented to be stable. One way of doing this is to artificially extend the key comparison so that comparisons between two objects with otherwise equal keys are decided using the order of the entries in the original input list as a tie-breaker. Remembering this order, however, may require additional time and space.

One application for stable sorting algorithms is sorting a list using a primary and secondary key. For example, suppose we wish to sort a hand of cards such that the suits are in the order clubs (♣), diamonds (♦), hearts (♥), spades (♠), and within each suit, the cards are sorted by rank. This can be done by first sorting the cards by rank (using any sort), and then doing a stable sort by suit:



Within each suit, the stable sort preserves the ordering by rank that was already done. This idea can be extended to any number of keys and is utilised by radix sort. The same effect can be achieved with an unstable sort by using a lexicographic key comparison, which, e.g., compares first by suit, and then compares by rank if the suits are the same.

Comparison of algorithms [edit]

This analysis assumes that the length of each key is constant and that all comparisons, swaps and other operations can proceed in constant time.

Legend:

- n is the number of records to be sorted.
- Comparison column has the following ranking classifications: "Best", "Average" and "Worst" if the time complexity is given for each case.
- "Memory" denotes the amount of additional storage required by the algorithm.
- The run times and the memory requirements listed are inside big O notation, hence the base of the logarithms does not matter.
- The notation $\log^2 n$ means $(\log n)^2$.

Comparison sorts [edit]

Below is a table of comparison sorts. Mathematical analysis demonstrates a comparison sort cannot perform better than $O(n \log n)$ on average.[4]

Comparison sorts

Name	Best	Average	Worst	Memory	Stable	In-place	Method	Other notes
Heapsort	$n \log n$	$n \log n$	$n \log n$	1	No	Yes	Selection	An optimized version of selection sort. Performs selection sort by constructing and maintaining a max heap to find the maximum in $O(\log n)$ time.
Introsort	$n \log n$	$n \log n$	$n \log n$	$\log n$	No	Yes	Partitioning & Selection	Used in several STL implementations. Performs a combination of Quicksort, Heapsort, and Insertion sort.
Merge sort	$n \log n$	$n \log n$	$n \log n$	n	Yes	No	Merging	Highly parallelizable (up to $O(\log n)$ using the Three Hungarians' Algorithm).[5]
In-Place Merge Sort	n	$n \log^2 n$	$n \log^2 n$	$\log n$	Yes	Yes	Merging	Variation of Mergesort which uses an $O(n \log n)$ in-place stable merge algorithm, such as rotate merge or symmerge.
Tournament sort	$n \log n$	$n \log n$	$n \log n$	n	Yes	No	Selection	An optimization of Selection Sort, which uses a tournament tree to select the min/max.
Tree sort	$n \log n$	$n \log n$	$n \log n$ (balanced)	n	Yes	No	Insertion	When using a self-balancing binary search tree.
Block sort	n	$n \log n$	$n \log n$	1	Yes	Yes	Insertion & Merging	Combine a block-based $O(n)$ in-place merge algorithm[6] with a bottom-up merge sort.
Smoothsort	n	$n \log n$	$n \log n$	1	No	Yes	Selection	Adaptive variant of heapsort based on the Leonardo sequence instead of a binary heap.
Timsort	n	$n \log n$	$n \log n$	n	Yes	No	Insertion & Merging	Makes $n - 1$ comparisons when the data is already sorted or reverse sorted.
Patience sorting	n	$n \log n$	$n \log n$	n	No	No	Insertion & Selection	Finds all the longest increasing subsequences in $O(n \log n)$.
Cubesort	n	$n \log n$	$n \log n$	n	Yes	No	Insertion	Makes $n - 1$ comparisons when the data is already sorted or reverse sorted.
Quicksort	$n \log n$	$n \log n$	n^2	$\log n$	No	Yes	Partitioning	Quicksort can be done in-place with $O(\log n)$ stack space.[7][8]
Fluxsort	n	$n \log n$	$n \log n$	n	Yes	No	Partitioning & Merging	An adaptive branchless stable introsort.
Crumsort	n	$n \log n$	$n \log n$	$\log n$	No	Yes	Partitioning & Merging	An in-place, but unstable variant of Fluxsort.
Library sort	$n \log n$	$n \log n$	n^2	n	No	No	Insertion	Similar to a gapped insertion sort.

Name	Best	Average	Worst	Memory	Stable	In-place	Method	Other notes
Shellsort	$n \log n$	$\Omega(n \log n)$	$O(n^{1+1/k})$ (Geometric) $n \log^2 n$ (Pratt)	1	No	Yes	Insertion	Small code size. Complexity is influenced by the gap sequence used. Pratt's sequence is worst case $\Theta(n \log^2 n)$ which is the best known. Tight bounds for the average and worst case remain open problems.
Comb sort	$n \log n$	n^2	n^2	1	No	Yes	Exchanging	Faster than bubble sort on average.
Insertion sort	n	n^2	n^2	1	Yes	Yes	Insertion	$O(n + d)$, in the worst case over sequences that have d inversions.
Bubble sort	n	n^2	n^2	1	Yes	Yes	Exchanging	Tiny code size.
Cocktail shaker sort	n	n^2	n^2	1	Yes	Yes	Exchanging	A bi-directional variant of Bubblesort.
Gnome sort	n	n^2	n^2	1	Yes	Yes	Exchanging	Tiny code size.
Odd–even sort	n	n^2	n^2	1	Yes	Yes	Exchanging	Can be run on parallel processors easily.
Strand sort	n	n^2	n^2	n	Yes	No	Selection	
Selection sort	n^2	n^2	n^2	1	No	Yes	Selection	Tiny code size. Noted for its simplicity and small number of element moves. Makes exactly $n - 1$ swaps.
Cycle sort	n^2	n^2	n^2	1	No	Yes	Selection	In-place with theoretically optimal number of writes.

Non-comparison sorts [edit]

The following table describes integer sorting algorithms and other sorting algorithms that are not comparison sorts. These algorithms are not limited to $\Omega(n \log n)$ unless meet unit-cost random-access machine model as described below.[9]

- Complexities below assume n items to be sorted, with keys of size k, digit size d, and r the range of numbers to be sorted.
- Many of them are based on the assumption that the key size is large enough that all entries have unique key values, and hence that $n \ll 2^k$, where $\ll$ means "much less than".
- In the unit-cost random-access machine model, algorithms with running time of $n \cdot \frac{k}{d}$, such as radix sort, still take time proportional to $\Theta(n \log n)$, because n is limited to be not more than $2^{\frac{k}{d}}$, and a larger number of elements to sort would require a bigger k in order to store them in the memory.[10]

Non-comparison sorts

Name	Best	Average	Worst	Memory	Stable	$n \ll 2^k$	Notes
Pigeonhole sort	—	$n + 2^k$	$n + 2^k$	2^k	Yes	Yes	Cannot sort non-integers.
Bucket sort (uniform keys)	—	$n + k$	$n^2 \cdot k$	$n \cdot k$	Yes	No	Assumes uniform distribution of elements from the domain in the array.[11] Also cannot sort non-integers.
Bucket sort (integer keys)	—	$n + r$	$n + r$	$n + r$	Yes	Yes	If r is $O(n)$, then average time complexity is $O(n)$.[12]
Counting sort	—	$n + r$	$n + r$	$n + r$	Yes	Yes	If r is $O(n)$, then average time complexity is $O(n)$.[11]
LSD Radix Sort	$n \cdot \frac{k}{d}$	$n \cdot \frac{k}{d}$	$n \cdot \frac{k}{d}$	$n + 2^d$	Yes	No	$\frac{k}{d}$ recursion levels, 2^d for count array.[11][12]

Name	Best	Average	Worst	Memory	Stable	$n \ll 2^k$	Notes
							Unlike most distribution sorts, this can sort non-integers.
MSD Radix Sort	n	$n \cdot \frac{k}{d}$	$n \cdot \frac{k}{d}$	$n + 2^d$	Yes	No	Stable version uses an external array of size n to hold all of the bins. Same as the LSD variant, it can sort non-integers.
MSD Radix Sort (in-place)	n	$n \cdot \frac{k}{1}$	$n \cdot \frac{k}{1}$	2^1	No	No	d=1 for in-place, $k/1$ recursion levels, no count array.
Spreadsort	n	$n \cdot \frac{k}{d}$	$n \cdot \left(\frac{k}{s} + d\right)$	$\frac{k}{d} \cdot 2^d$	No	No	Asymptotic are based on the assumption that $n \ll 2^k$, but the algorithm does not require this.
Burstsort	—	$n \cdot \frac{k}{d}$	$n \cdot \frac{k}{d}$	$n \cdot \frac{k}{d}$	No	No	Has better constant factor than radix sort for sorting strings. Though relies somewhat on specifics of commonly encountered strings.
Flashsort	n	$n + r$	n^2	n	No	No	Requires uniform distribution of elements from the domain in the array to run in linear time. If distribution is extremely skewed then it can go quadratic if underlying sort is quadratic (it is usually an insertion sort). In-place version is not stable.

Samplesort can be used to parallelize any of the non-comparison sorts, by efficiently distributing data into several buckets and then passing down sorting to several processors, with no need to merge as buckets are already sorted between each other.

Others [edit]

Some algorithms are slow compared to those discussed above, such as the bogosort with unbounded run time and the stooge sort which has $O(n^{2.7})$ run time. These sorts are usually described for educational purposes to demonstrate how the run time of algorithms is estimated. The following table describes some sorting algorithms that are impractical for real-life use in traditional software contexts due to extremely poor performance or specialized hardware requirements.

Name	Best	Average	Worst	Memory	Stable	Comparison	Other notes
Bead sort	n	S	S	n^2	—	No	Works only with positive integers. Requires specialized hardware for it to run in guaranteed $O(n)$ time. There is a possibility for software implementation, but running time will be $O(S)$, where S is the sum of all integers to be sorted; in the case of small integers, it can be considered to be linear.
Merge-insertion sort	$n \log n$ comparisons	$n \log n$ comparisons	$n \log n$ comparisons	Varies	No	Yes	Makes very few comparisons worst case compared to other sorting algorithms. Mostly of theoretical interest due to implementational complexity and suboptimal data moves.
Spaghetti (Poll) sort	n	n	n	n^2	Yes	Polling	This is a linear-time, analog algorithm for sorting a sequence of items, requiring $O(n)$ stack space, and the sort is stable. This requires n parallel

Name	Best	Average	Worst	Memory	Stable	Comparison	Other notes
							processors. See spaghetti sort § Analysis.
Sorting network	Varies	Varies	Varies	Varies	Varies (stable sorting networks require more comparisons)	Yes	Order of comparisons are set in advance based on a fixed network size.
Bitonic sorter	$\log^2 n$ parallel	$\log^2 n$ parallel	$n \log^2 n$ non-parallel	1	No	Yes	An effective variation of Sorting networks. [*disputed – discuss*]
Bogosort	n	$(n \times n!)$	Unbounded	1	No	Yes	Random shuffling. Used for example purposes only, as even the expected best-case runtime is awful.[13] Worst case is unbounded when using randomization, but a deterministic version guarantees $O(n \times n!)$ worst case.
Stooge sort	$n^{\log 3/\log 1.5}$	$n^{\log 3/\log 1.5}$	$n^{\log 3/\log 1.5}$	$\log n$	No	Yes	Slower than most of the sorting algorithms (even naive ones) with a time complexity of $O(n^{\log 3 / \log 1.5}) = O(n^{2.7095\ldots})$ Can be made stable, and is also a sorting network.
Slowsort	$o\left(n^{\log_2(n)/2}\right)$	$o\left(n^{\log_2(n)/2}\right)$	$o\left(n^{\log_2(n)/2}\right)$	n	No	Yes	A multiply and surrender algorithm, antonymous with divide-and-conquer algorithm.

Theoretical computer scientists have invented other sorting algorithms that provide better than $O(n \log n)$ time complexity assuming certain constraints, including:

- Thorup's algorithm,[14] a randomized integer sorting algorithm, taking $O(n \log \log n)$ time and $O(n)$ space.[14]
- AHNR algorithm,[15] an integer sorting algorithm which runs in $O(n \log \log n)$ time deterministically, and also has a randomized version which runs in linear time when words are large enough, specifically $w \geq (\log n)^{2+\varepsilon}$ (where w is the word size).
- A randomized integer sorting algorithm taking $O\left(n\sqrt{\log \log n}\right)$ expected time and $O(n)$ space.[16]

Popular sorting algorithms [edit]

While there are a large number of sorting algorithms, in practical implementations a few algorithms predominate. Insertion sort is widely used for small data sets, while for large data sets an asymptotically efficient sort is used, primarily heapsort, merge sort, or quicksort. Efficient implementations generally use a hybrid algorithm, combining an asymptotically efficient algorithm for the overall sort with insertion sort for small lists at the bottom of a recursion. Highly tuned implementations use more sophisticated variants, such as Timsort (merge sort, insertion sort, and additional logic), used in Android, Java, and Python, and introsort (quicksort and heapsort), used (in variant forms) in some C++ sort implementations and in .NET.

For more restricted data, such as numbers in a fixed interval, distribution sorts such as counting sort or radix sort are widely used. Bubble sort and variants are rarely used in practice, but are commonly found in teaching and theoretical discussions.

When physically sorting objects (such as alphabetizing papers, tests or books) people intuitively generally use insertion sorts for small sets. For larger sets, people often first bucket, such as by initial letter, and multiple bucketing allows practical sorting of very large sets. Often space is relatively cheap, such as by spreading objects out on the floor or over a large area, but operations are expensive, particularly moving an object a large distance – locality of reference is important. Merge sorts are also practical for physical objects, particularly as two hands can be used, one for each list to merge, while other algorithms, such as heapsort or quicksort, are poorly suited for human use. Other algorithms, such as library sort, a variant of insertion sort that leaves spaces, are also practical for physical use.

Simple sorts [edit]

Two of the simplest sorts are insertion sort and selection sort, both of which are efficient on small data, due to low overhead, but not efficient on large data. Insertion sort is generally faster than selection sort in practice, due to fewer comparisons and good performance on almost-sorted data, and thus is preferred in practice, but selection sort uses fewer writes, and thus is used when write performance is a limiting factor.

Insertion sort [edit]

Main article: Insertion sort

Insertion sort is a simple sorting algorithm that is relatively efficient for small lists and mostly sorted lists, and is often used as part of more sophisticated algorithms. It works by taking elements from the list one by one and inserting them in their correct position into a new sorted list similar to how one puts money in their wallet.[17] In arrays, the new list and the remaining elements can share the array's space, but insertion is expensive, requiring shifting all following elements over by one. Shellsort is a variant of insertion sort that is more efficient for larger lists.

Selection sort [edit]

Main article: Selection sort

Selection sort is an in-place comparison sort. It has $O(n^2)$ complexity, making it inefficient on large lists, and generally performs worse than the similar insertion sort. Selection sort is noted for its simplicity and also has performance advantages over more complicated algorithms in certain situations.

The algorithm finds the minimum value, swaps it with the value in the first position, and repeats these steps for the remainder of the list.[18] It does no more than n swaps and thus is useful where swapping is very expensive.

Efficient sorts [edit]

Practical general sorting algorithms are almost always based on an algorithm with average time complexity (and generally worst-case complexity) $O(n \log n)$, of which the most common are heapsort, merge sort, and quicksort. Each has advantages and drawbacks, with the most significant being that simple implementation of merge sort uses $O(n)$ additional space, and simple implementation of quicksort has $O(n^2)$ worst-case complexity. These problems can be solved or ameliorated at the cost of a more complex algorithm.

While these algorithms are asymptotically efficient on random data, for practical efficiency on real-world data various modifications are used. First, the overhead of these algorithms becomes significant on smaller data, so often a hybrid algorithm is used, commonly switching to insertion sort once the data is small enough. Second, the algorithms often perform poorly on already sorted data or almost sorted data – these are common in real-world data and can be sorted in $O(n)$ time by appropriate algorithms. Finally, they may also be unstable, and stability is often a desirable property in a sort. Thus more sophisticated algorithms are often employed, such as Timsort (based on merge sort) or introsort (based on quicksort, falling back to heapsort).

Merge sort [edit]

Main article: Merge sort

Merge sort takes advantage of the ease of merging already sorted lists into a new sorted list. It starts by comparing every two elements (i.e., 1 with 2, then 3 with 4...) and swapping them if the first should come after the second. It then merges each of the resulting lists of two into lists of four, then merges those lists of four, and so on; until at last two lists are merged into the final sorted list.[19] Of the algorithms described here, this is the first that scales well to very large lists, because its worst-case running time is $O(n \log n)$. It is also easily applied to lists, not only arrays, as it only requires sequential access, not random access. When sorting arrays, it has additional $O(n)$ space complexity, and involves a large number of copies in simple implementations; however, linked lists can be merge sorted with constant extra space, as such it is the algorithm of choice for sorting linked lists.

Merge sort has seen a relatively recent surge in popularity for practical implementations, due to its use in the sophisticated algorithm Timsort, which is used for the standard sort routine in the programming languages Python[20] and Java (as of JDK7[21]). Merge sort itself is the standard routine in Perl,[22] among others, and has been used in Java at least since 2000 in JDK1.3.[23]

Heapsort [edit]

Main article: Heapsort

Heapsort is a much more efficient version of selection sort. It also works by determining the largest (or smallest) element of the list, placing that at the end (or beginning) of the list, then continuing with the rest of the list, but accomplishes this task efficiently by using a data structure called a heap, a special type of binary tree.[24] Once the data list has been made into a heap, the root node is guaranteed to be the largest (or smallest) element. When it is removed and placed at the end of the list, the heap is rearranged so the largest element remaining moves to the root. Using

the heap, finding the next largest element takes O(log *n*) time, instead of O(*n*) for a linear scan as in simple selection sort. This allows Heapsort to run in O(*n* log *n*) time, and this is also the worst-case complexity.

Quicksort [edit]

Main article: Quicksort

Quicksort is a divide-and-conquer algorithm which relies on a *partition* operation: to partition an array, an element called a *pivot* is selected.[25][26] All elements smaller than the pivot are moved before it and all greater elements are moved after it. This can be done efficiently in linear time and in-place. The lesser and greater sublists are then recursively sorted. This yields an average time complexity of O(*n* log *n*), with low overhead, and thus this is a popular algorithm. Efficient implementations of quicksort (with in-place partitioning) are typically unstable sorts and somewhat complex but are among the fastest sorting algorithms in practice. Together with its modest O(log *n*) space usage, quicksort is one of the most popular sorting algorithms and is available in many standard programming libraries.

The important caveat about quicksort is that its worst-case performance is O(n^2); while this is rare, in naive implementations (choosing the first or last element as pivot) this occurs for sorted data, which is a common case. The most complex issue in quicksort is thus choosing a good pivot element, as consistently poor choices of pivots can result in drastically slower O(n^2) performance, but good choice of pivots yields O(*n* log *n*) performance, which is asymptotically optimal. For example, if at each step the median is chosen as the pivot then the algorithm works in O(*n* log *n*). Finding the median, such as by the median of medians selection algorithm is however an O(*n*) operation on unsorted lists and therefore exacts significant overhead with sorting. In practice choosing a random pivot almost certainly yields O(*n* log *n*) performance.

If a guarantee of O(*n* log *n*) performance is important, there is a simple modification to achieve that. The idea, due to Musser, is to set a limit on the maximum depth of recursion.[27] If that limit is exceeded, then sorting is continued using the heapsort algorithm. Musser proposed that the limit should be $1 + 2\lfloor \log_2(n) \rfloor$, which is approximately twice the maximum recursion depth one would expect on average with a randomly ordered array.

Shellsort [edit]

Main article: Shellsort

A Shellsort, different from bubble sort in that it moves elements to numerous swapping positions.

Shellsort was invented by Donald Shell in 1959.[28] It improves upon insertion sort by moving out of order elements more than one position at a time. The concept behind Shellsort is that insertion sort performs in $O(kn)$ time, where k is the greatest distance between two out-of-place elements. This means that generally, they perform in O(n^2), but for data that is mostly sorted, with only a few elements out of place, they perform faster. So, by first sorting elements far away, and progressively shrinking the gap between the elements to sort, the final sort computes much faster. One implementation can be described as arranging the data sequence in a two-dimensional array and then sorting the columns of the array using insertion sort.

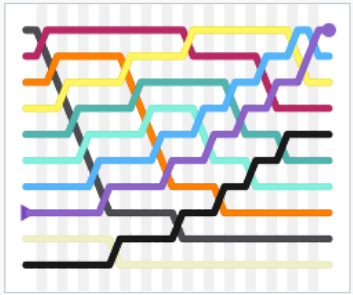
The worst-case time complexity of Shellsort is an open problem and depends on the gap sequence used, with known complexities ranging from O(n^2) to O($n^{4/3}$) and Θ($n \log^2 n$). This, combined with the fact that Shellsort is in-place, only needs a relatively small amount of code, and does not require use of the call stack, makes it is useful in situations where memory is at a premium, such as in embedded systems and operating system kernels.

Bubble sort and variants [edit]

Bubble sort, and variants such as the Comb sort and cocktail sort, are simple, highly inefficient sorting algorithms. They are frequently seen in introductory texts due to ease of analysis, but they are rarely used in practice.

Bubble sort [edit]

Main article: Bubble sort



A bubble sort, a sorting algorithm that continuously steps through a list,

Bubble sort is a simple sorting algorithm. The algorithm starts at the beginning of the data set. It compares the first two elements, and if the first is greater than the second, it swaps them. It continues doing this for each pair of adjacent elements to the end of the data set. It then starts again with the first two elements, repeating until no swaps have occurred on the last pass.[29] This algorithm's average time and worst-case performance is O(n^2), so it is rarely used to sort large, unordered data sets. Bubble sort can be used to sort a small number of items (where its asymptotic inefficiency is not a high penalty). Bubble sort can also be used efficiently on a list of any length that is nearly sorted (that is, the elements are not significantly out of place). For example, if any number of elements are out of place by only one

position (e.g. 0123546789 and 1032547698), bubble sort's exchange will get them in order on the first pass, the second pass will find all elements in order, so the sort will take only 2*n* time.

swapping items until they appear in the correct order.

Comb sort [edit]

Main article: Comb sort

Comb sort is a relatively simple sorting algorithm based on bubble sort and originally designed by Włodzimierz Dobosiewicz in 1980.[30] It was later rediscovered and popularized by Stephen Lacey and Richard Box with a *Byte* Magazine article published in April 1991. The basic idea is to eliminate *turtles*, or small values near the end of the list, since in a bubble sort these slow the sorting down tremendously. (*Rabbits*, large values around the beginning of the list, do not pose a problem in bubble sort) It accomplishes this by initially swapping elements that are a certain distance from one another in the array, rather than only swapping elements if they are adjacent to one another, and then shrinking the chosen distance until it is operating as a normal bubble sort. Thus, if Shellsort can be thought of as a generalized version of insertion sort that swaps elements spaced a certain distance away from one another, comb sort can be thought of as the same generalization applied to bubble sort.

Distribution sorts [edit]

See also: External sorting

Distribution sort refers to any sorting algorithm where data is distributed from their input to multiple intermediate structures which are then gathered and placed on the output. For example, both bucket sort and flashsort are distribution-based sorting algorithms. Distribution sorting algorithms can be used on a single processor, or they can be a distributed algorithm, where individual subsets are separately sorted on different processors, then combined. This allows external sorting of data too large to fit into a single computer's memory.

Counting sort [edit]

Main article: Counting sort

Counting sort is applicable when each input is known to belong to a particular set, S, of possibilities. The algorithm runs in $O(|S| + n)$ time and $O(|S|)$ memory where n is the length of the input. It works by creating an integer array of size $|S|$ and using the ith bin to count the occurrences of the ith member of S in the input. Each input is then counted by incrementing the value of its corresponding bin. Afterward, the counting array is looped through to arrange all of the inputs in order. This sorting algorithm often cannot be used because S needs to be reasonably small for the algorithm to be efficient, but it is extremely fast and demonstrates great asymptotic behavior as n increases. It also can be modified to provide stable behavior.

Bucket sort [edit]

Main article: Bucket sort

Bucket sort is a divide-and-conquer sorting algorithm that generalizes counting sort by partitioning an array into a finite number of buckets. Each bucket is then sorted individually, either using a different sorting algorithm or by recursively applying the bucket sorting algorithm.

A bucket sort works best when the elements of the data set are evenly distributed across all buckets.

Radix sort [edit]

Main article: Radix sort

Radix sort is an algorithm that sorts numbers by processing individual digits. n numbers consisting of k digits each are sorted in $O(n \cdot k)$ time. Radix sort can process digits of each number either starting from the least significant digit (LSD) or starting from the most significant digit (MSD). The LSD algorithm first sorts the list by the least significant digit while preserving their relative order using a stable sort. Then it sorts them by the next digit, and so on from the least significant to the most significant, ending up with a sorted list. While the LSD radix sort requires the use of a stable sort, the MSD radix sort algorithm does not (unless stable sorting is desired). In-place MSD radix sort is not stable. It is common for the counting sort algorithm to be used internally by the radix sort. A hybrid sorting approach, such as using insertion sort for small bins, improves performance of radix sort significantly.

Table of running times of popular algorithms [edit]

In *Algorithms and Data Structures* Niklaus Wirth gives a comparison of the running time of several of the popular algorithms on the Lilith computer.[31]

Sorting 2048 random items

Algorithm	Time (seconds)
Bubble sort	128.84

Shaker sort	104.44
Selection sort	58.34
Insertion sort	50.74
Binary insertion sort	37.66
Shell sort	7.08
Heap sort	2.22
Merge sort	2.06
Non-recursive quicksort	1.32
Recursive quicksort	1.22

Memory usage patterns and index sorting [edit]

When the size of the array to be sorted approaches or exceeds the available primary memory, so that (much slower) disk or swap space must be employed, the memory usage pattern of a sorting algorithm becomes important, and an algorithm that might have been fairly efficient when the array fit easily in RAM may become impractical. In this scenario, the total number of comparisons becomes (relatively) less important, and the number of times sections of memory must be copied or swapped to and from the disk can dominate the performance characteristics of an algorithm. Thus, the number of passes and the localization of comparisons can be more important than the raw number of comparisons, since comparisons of nearby elements to one another happen at system bus speed (or, with caching, even at CPU speed), which, compared to disk speed, is virtually instantaneous.

For example, the popular recursive quicksort algorithm provides quite reasonable performance with adequate RAM, but due to the recursive way that it copies portions of the array it becomes much less practical when the array does not fit in RAM, because it may cause a number of slow copy or move operations to and from disk. In that scenario, another algorithm may be preferable even if it requires more total comparisons.

One way to work around this problem, which works well when complex records (such as in a relational database) are being sorted by a relatively small key field, is to create an index into the array and then sort the index, rather than the entire array. (A sorted version of the entire array can then be produced with one pass, reading from the index, but often even that is unnecessary, as having the sorted index is adequate.) Because the index is much smaller than the entire array, it may fit easily in memory where the entire array would not, effectively eliminating the disk-swapping problem. This procedure is sometimes called "tag sort".[32]

Another technique for overcoming the memory-size problem is using external sorting, for example, one of the ways is to combine two algorithms in a way that takes advantage of the strength of each to improve overall performance. For instance, the array might be subdivided into chunks of a size that will fit in RAM, the contents of each chunk sorted using an efficient algorithm (such as quicksort), and the results merged using a *k*-way merge similar to that used in merge sort. This is faster than performing either merge sort or quicksort over the entire list.[33][34]

Techniques can also be combined. For sorting very large sets of data that vastly exceed system memory, even the index may need to be sorted using an algorithm or combination of algorithms designed to perform reasonably with virtual memory, i.e., to reduce the amount of swapping required.

Related algorithms [edit]

Related problems include approximate sorting (sorting a sequence to within a certain amount of the correct order), partial sorting (sorting only the *k* smallest elements of a list, or finding the *k* smallest elements, but unordered) and selection (computing the *k*th smallest element). These can be solved inefficiently by a total sort, but more efficient algorithms exist, often derived by generalizing a sorting algorithm. The most notable example is quickselect, which is related to quicksort. Conversely, some sorting algorithms can be derived by repeated application of a selection algorithm; quicksort and quickselect can be seen as the same pivoting move, differing only in whether one recurses on both sides (quicksort, divide-and-conquer) or one side (quickselect, decrease-and-conquer).

A kind of opposite of a sorting algorithm is a shuffling algorithm. These are fundamentally different because they require a source of random numbers. Shuffling can also be implemented by a sorting algorithm, namely by a random sort: assigning a random number to each element of the list and then sorting based on the random numbers. This is generally not done in practice, however, and there is a well-known simple and efficient algorithm for shuffling: the Fisher–Yates shuffle.

Sorting algorithms are ineffective for finding an order in many situations. Usually, when elements have no reliable comparison function (crowdsourced preferences like voting systems), comparisons are very costly (sports), or when it would be impossible to pairwise compare all elements for all criteria (search engines). In these cases, the problem is usually referred to as *ranking* and the goal is to find the "best" result for

some criteria according to probabilities inferred from comparisons or rankings. A common example is in chess, where players are ranked with the Elo rating system, and rankings are determined by a tournament system instead of a sorting algorithm.

There are sorting algorithms for a "noisy" (potentially incorrect) comparator and sorting algorithms for a pair of "fast and dirty" (i.e. "noisy") and "clean" comparators. This can be useful when the full comparison function is costly.[35]

See also [edit]

- Collation – Assembly of written information into a standard order
- K-sorted sequence
- Schwartzian transform – Programming idiom for efficiently sorting a list by a computed key
- Search algorithm – Any algorithm which solves the search problem
- Quantum sort – Sorting algorithms for quantum computers

References [edit]

1. ^ "Meet the 'Refrigerator Ladies' Who Programmed the ENIAC". *Mental Floss*. 2013-10-13. Archived from the original on 2018-10-08. Retrieved 2016-06-16.
2. ^ Lohr, Steve (Dec 17, 2001). "Frances E. Holberton, 84, Early Computer Programmer". NYTimes. Archived from the original on 16 December 2014. Retrieved 16 December 2014.
3. ^ Demuth, Howard B. (1956). *Electronic Data Sorting* (PhD thesis). Stanford University. ProQuest 301940891.
4. ^ Cormen, Thomas H.; Leiserson, Charles E.; Rivest, Ronald L.; Stein, Clifford (2009), "8", *Introduction To Algorithms* (3rd ed.), Cambridge, MA: The MIT Press, p. 167, ISBN 978-0-262-03293-3
5. ^ Ajtai, M.; Komlós, J.; Szemerédi, E. (1983). *An $O(n \log n)$ sorting network*. STOC '83. *Proceedings of the fifteenth annual ACM symposium on Theory of computing*. pp. 1–9. doi:10.1145/800061.808726. ISBN 0-89791-099-0.
6. ^ Kim, P. S.; Kutzner, A. (2008). *Ratio Based Stable In-Place Merging*. TAMC 2008. *Theory and Applications of Models of Computation*. LNCS. Vol. 4978. pp. 246–257. CiteSeerX 10.1.1.330.2641. doi:10.1007/978-3-540-79228-4_22. ISBN 978-3-540-79227-7.
7. ^ Sedgewick, Robert (1 September 1998). *Algorithms In C: Fundamentals, Data Structures, Sorting, Searching, Parts 1-4* (3 ed.). Pearson Education. ISBN 978-81-317-1291-7. Retrieved 27 November 2012.
8. ^ Sedgewick, R. (1978). "Implementing Quicksort programs". *Comm. ACM*. **21** (10): 847–857. doi:10.1145/359619.359631. S2CID 10020756.
9. ^ Cormen, Thomas H.; Leiserson, Charles E.; Rivest, Ronald L.; Stein, Clifford (2001), "8", *Introduction To Algorithms* (2nd ed.), Cambridge, MA: The MIT Press, p. 165, ISBN 0-262-03293-7
10. ^ Nilsson, Stefan (2000). "The Fastest Sorting Algorithm?". *Dr. Dobb's*. Archived from the original on 2019-06-08. Retrieved 2015-11-23.
11. ^ [a] [b] [c] Cormen, Thomas H.; Leiserson, Charles E.; Rivest, Ronald L.; Stein, Clifford (2001) [1990]. *Introduction to Algorithms* (2nd ed.). MIT Press and McGraw-Hill. ISBN 0-262-03293-7.
12. ^ [a] [b] Goodrich, Michael T.; Tamassia, Roberto (2002). "4.5 Bucket-Sort and Radix-Sort". *Algorithm Design: Foundations, Analysis, and Internet Examples*. John Wiley & Sons. pp. 241–243. ISBN 978-0-471-38365-9.
13. ^ Gruber, H.; Holzer, M.; Ruepp, O. (2007), "Sorting the slow way: an analysis of perversely awful randomized sorting algorithms", *4th International Conference on Fun with Algorithms, Castiglioncello, Italy, 2007* (PDF), Lecture Notes in Computer Science, vol. 4475, Springer-Verlag, pp. 183–197, doi:10.1007/978-3-540-72914-3_17, ISBN 978-3-540-72913-6, archived (PDF) from the original on 2020-09-29, retrieved 2020-06-27.
14. ^ [a] [b] Thorup, M. (February 2002). "Randomized Sorting in O(n log log n) Time and Linear Space Using Addition, Shift, and Bit-wise Boolean Operations". *Journal of Algorithms*. **42** (2): 205–230. doi:10.1006/jagm.2002.1211. S2CID 9700543.
15. ^ Andersson, Arne; Hagerup, Torben; Nilsson, Stefan; Raman, Rajeev (1995). "Sorting in linear time?". *Proceedings of the twenty-seventh annual ACM symposium on Theory of computing*. ACM. pp. 427–436.
16. ^ Han, Yijie; Thorup, M. (2002). *Integer sorting in $O(n\sqrt{(\log \log n)})$ expected time and linear space*. The 43rd Annual IEEE Symposium on Foundations of Computer Science. pp. 135–144. doi:10.1109/SFCS.2002.1181890. ISBN 0-7695-1822-2.
17. ^ Wirth, Niklaus (1986). *Algorithms & Data Structures*. Upper Saddle River, NJ: Prentice-Hall. pp. 76–77. ISBN 978-0130220059.
18. ^ Wirth 1986, pp. 79–80
19. ^ Wirth 1986, pp. 101–102
20. ^ "Tim Peters's original description of timsort". *python.org*. Archived from the original on 22 January 2018. Retrieved 14 April 2018.
21. ^ "OpenJDK's TimSort.java". *java.net*. Archived from the original on 14 August 2011. Retrieved 14 April 2018.
22. ^ "sort – perldoc.perl.org". *perldoc.perl.org*. Archived from the original on 14 April 2018. Retrieved 14 April 2018.
23. ^ Merge sort in Java 1.3, Sun. Archived 2009-03-04 at the Wayback Machine
24. ^ Wirth 1986, pp. 87–89
25. ^ Wirth 1986, p. 93
26. ^ Cormen, Thomas H.; Leiserson, Charles E.; Rivest, Ronald L.; Stein, Clifford (2009), *Introduction to Algorithms* (3rd ed.), Cambridge, MA: The MIT Press, pp. 171–172, ISBN 978-0262033848
27. ^ Musser, David R. (1997), "Introspective Sorting and Selection Algorithms", *Software: Practice and Experience*, **27** (8): 983–993, doi:10.1002/(SICI)1097-024X(199708)27:8<983::AID-SPE117>3.0.CO;2-#
28. ^ Shell, D. L. (1959). "A High-Speed Sorting Procedure" (PDF). *Communications of the ACM*. **2** (7): 30–32. doi:10.1145/368370.368387. S2CID 28572656. Archived from the original (PDF) on 2017-08-30. Retrieved 2020-03-23.
29. ^ Wirth 1986, pp. 81–82
30. ^ Brejová, B. (15 September 2001). "Analyzing variants of Shellsort". *Inf. Process. Lett.* **79** (5): 223–227. doi:10.1016/S0020-0190(00)00223-4.
31. ^ Wirth 1986, p. 100.
32. ^ "tag sort Definition from PC Magazine Encyclopedia". *Pcmag.com*. Archived from the original on 6 October 2012. Retrieved 14 April 2018.

33. ^ Donald Knuth, *The Art of Computer Programming*, Volume 3: *Sorting and Searching*, Second Edition. Addison-Wesley, 1998, ISBN 0-201-89685-0, Section 5.4: External Sorting, pp. 248–379.
34. ^ Ellis Horowitz and Sartaj Sahni, *Fundamentals of Data Structures*, H. Freeman & Co., ISBN 0-7167-8042-9.
35. ^ Bai, Xingjian; Coester, Christian (2023). *Sorting with Predictions*. NeurIPS. p. 5.

Further reading [edit]

- Knuth, Donald E. (1998), *Sorting and Searching*, The Art of Computer Programming, vol. 3 (2nd ed.), Boston: Addison-Wesley, ISBN 0-201-89685-0
- Sedgewick, Robert (1980), "Efficient Sorting by Computer: An Introduction", *Computational Probability*, New York: Academic Press, pp. 101–130, ISBN 0-12-394680-8

External links [edit]

- Sorting Algorithm Animations at the Wayback Machine (archived 3 March 2015).
- Sequential and parallel sorting algorithms – Explanations and analyses of many sorting algorithms.
- Dictionary of Algorithms, Data Structures, and Problems – Dictionary of algorithms, techniques, common functions, and problems.
- Slightly Skeptical View on Sorting Algorithms – Discusses several classic algorithms and promotes alternatives to the quicksort algorithm.
- 15 Sorting Algorithms in 6 Minutes (Youtube) – Visualization and "audibilization" of 15 Sorting Algorithms in 6 Minutes.
- A036604 sequence in OEIS database titled "Sorting numbers: minimal number of comparisons needed to sort n elements" – Performed by Ford–Johnson algorithm.
- Sorting Algorithms Used on Famous Paintings (Youtube) – Visualization of Sorting Algorithms on Many Famous Paintings.
- A Comparison of Sorting Algorithms – Runs a series of tests of 9 of the main sorting algorithms using Python timeit and Google Colab.

Categories: Sorting algorithms | Data processing